

PlaceHub

Placement Portal Application

MAD-2 Project Report · IITM BS Degree Programme

1. Student Details

Name	Sanket Garg
Roll Number	23f3001457
Email	23f3001457@ds.study.iitm.ac.in
About Me	I am a B.Tech student at Jabalpur Engineering College also enrolled in the IIT Madras BS Degree Programme, with a keen interest in full-stack web development and building practical, real-world applications.

2. Project Details

Project Title	PlaceHub — Placement Portal Application
Problem Statement	To design and build a multi-role web application that manages the complete campus placement lifecycle — from company registration and drive creation to student applications, interview scheduling, and result tracking.
Approach	The app was built using Flask as the backend framework with a REST API structure. It supports three roles — Admin, Company, and Student — each with a dedicated dashboard. Redis is used for caching and as a Celery broker for background jobs like email reminders, monthly reports, and CSV exports. The frontend is built with Vue.js 3 via CDN with a professional sidebar UI.

3. AI/LLM Declaration

I used Claude and Gemini to assist in writing code, debugging errors and designing the UI. The extent of AI/LLM usage is approximately 15–20%, limited to code suggestions, UI design ideas, and debugging assistance.

All final logic, integration, and feature decisions were reviewed and understood by me. The overall project architecture and feature planning were done by me.

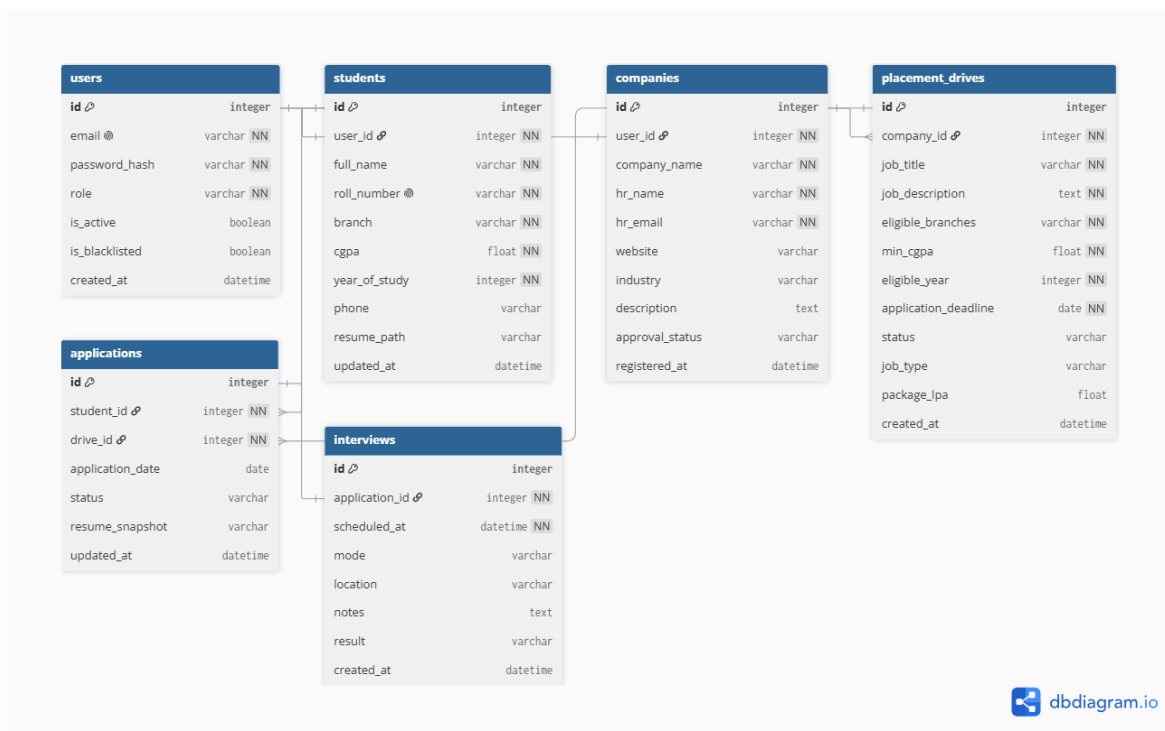
4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend REST API framework
SQLAlchemy	Object Relational Mapper for SQLite database
Flask-JWT-Extended	JWT-based authentication for all roles
Redis	Caching + Celery message broker

Technology / Library	Purpose
Celery	Asynchronous background jobs
Flask-Mail	Email reminders and monthly reports
Vue.js 3 (CDN)	Single Page Application frontend
Vue Router	Client-side routing with navigation guard
Chart.js	Admin dashboard charts
SQLite	Local database for all data storage
Bootstrap 5	Responsive UI styling and components
Flask-Caching	API response caching with Redis backend

5. Database Schema / ER Diagram

•



6. API Resource Endpoints

Endpoint	Method	Description
/api/auth/login	POST	Login for all roles
/api/auth/register/student	POST	Register student
/api/auth/register/company	POST	Register company
/api/admin/dashboard	GET	Admin dashboard stats
/api/admin/students	GET	List all students
/api/admin/students/{id}/blacklist	PATCH	Blacklist/reactivate student
/api/admin/companies	GET	List all companies
/api/admin/companies/{id}/approve	PATCH	Approve company
/api/admin/companies/{id}/reject	PATCH	Reject company
/api/admin/drives	GET	List all drives
/api/admin/drives/{id}/approve	PATCH	Approve drive
/api/admin/drives/{id}/close	PATCH	Mark drive complete
/api/admin/applications	GET	List all applications
/api/admin/search	GET	Search students and companies
/api/company/drives	POST	Create placement drive
/api/company/drives/{id}/applications	GET	List applicants
/api/company/applications/{id}/status	PATCH	Update applicant status
/api/company/applications/{id}/interview	POST	Schedule interview
/api/student/drives	GET	Browse drives
/api/student/drives/{id}/apply	POST	Apply to drive
/api/student/applications	GET	My applications
/api/student/profile/resume	POST	Upload resume
/api/student/export/csv	POST	Export applications as CSV

7. Architecture and Features

Architecture Overview:

- app.py — Main Flask application entry point (application factory pattern)
- config.py — Environment-based configuration (secrets via .env)
- models.py — Database models using SQLAlchemy ORM
- routes/ — Flask Blueprints for auth, admin, company, and student routes
- tasks/ — Celery tasks for reminders, reports, and CSV export
- frontend/index.html — Jinja2 HTML entry point (SPA shell)

- frontend/js/ — Vue.js 3 components and views (CDN, no build step)
- frontend/css/ — Custom design system on top of Bootstrap 5

Implemented Features:

- Student and company registration and login (JWT-based auth)
- Admin approval workflow for companies and drives
- Student eligibility filtering for placement drives (branch, CGPA, year)
- Interview scheduling and result tracking with email notifications
- Redis caching on admin dashboard (5-minute TTL, cache invalidation on update)
- Async CSV export using Celery (poll status → download)
- Daily reminder and monthly report emails via Flask-Mail + Gmail SMTP
- Chart.js visualizations — doughnut (application status) + bar (students by branch)
- Resume upload and download (PDF, max 5MB, secure_filename enforced)
- Frontend and backend form validation
- Role-based access control with Vue Router navigation guards
- Duplicate application prevention (DB UniqueConstraint + API check)
- Responsive sidebar layout (AppShell component) with mobile support

8. Video Presentation

Drive Link:

 [Mad-2 Project Video.mp4](#)

Thank You